Intro to Data Visualization

This document contains useful information on how to use Python visualization libraries to create intuitive and insightful visuals.

Note: The terminology included in this document and knowledge of data visualization is course content and therefore may be included on exams or quizzes.

Table of Contents

Introduction **2**
- Terminology 2
- Getting Started 3

Visualization Fundamentals **3**
- What is a visualization? 3
- What is human perception? 3
- What makes a good visualization? 4

Libraries **5**
- Matplotlib 5
- Seaborn 5
- Plotly 5
- Which one should you use? 5

Examples **6**
- **Scatter** **6**
  - Matplotlib 7
  - Seaborn 8
  - Plotly 9
- **Line** **10**
  - Matplotlib 11
  - Seaborn 12
  - Plotly 13
- **Bar** **14**
  - Matplotlib 15
  - Seaborn 16
  - Plotly 17
- **Pie Chart** **18**
  - Matplotlib 18
  - Seaborn 18
  - Plotly 19
- Histogram 20
  - Matplotlib 20
  - Seaborn 22
  - Plotly 23
- **Box** **25**
  - Matplotlib 26
  - Seaborn 27

Plotly 28
More **28**

Introduction

Data visualization is used to communicate ideas and insights. Additionally, data visualization is often a great tool for data exploration, data analysis, and data presentation. It can be used by teachers to display student test results, by computer scientists exploring advancements in their research, or by executives looking to share information with stakeholders. Data visualization provides a quick and effective way to communicate information in a universal manner using visual information.

Terminology

Data Visualization - the discipline of trying to understand data by placing it in a visual context so that patterns, trends and correlations that might not otherwise be detected can be exposed. [1]
Human Perception - the process of recognizing , organizing , and interpreting sensory information. This includes visual information in the form of images or charts.
Quantitative - data that can be measured on a numerical scale. Examples of such data are length, height, speed, population, cost.
Qualitative - non-numeric information that doesn't involve measurements or quantities. Examples of such data are eye color, level of education, non-numerical ratings (poor, good, great)
Figure - the outermost container for a matplotlib graphic, which can contain multiple Axes objects.
Axes - in matplotlib, one individual plot or graph within a figure. In other plotting libraries, axes is the plural of axis and represents the directions being plotted.
Legend - a guide to the data that appears on a chart. A legend works like a key to the colors and patterns used to represent the data, and is particularly useful if the chart includes multiple datasets.

Getting Started

By convention, the matplotlib module is imported with the following line of code: `import matplotlib.pyplot as plt`. Matplotlib should have been installed with Anaconda or Miniconda, however if the import throws an error, execute `conda install matplotlib` from

[1] Tanner, Gilbert. “Introduction to Data Visualization in Python.” *Medium*, Towards Data Science, 6 Mar. 2019

the command line. Similarly, the conventional imports for seaborn and plotly are `import seaborn as sns` and `import plotly.graph_objects as go` (or `import plotly.express as px`) accordingly. If either import throws an error execute `conda install seaborn` and `conda install plotly` from the command line. When adding a trendline to a plotly visualization as shown in a later example, you might need to add the statsmodels statistical analysis package as well. You can install that by doing `conda install statsmodels` from the command line too.

Visualization Fundamentals

At the core of a great visualization you will find techniques that leverage human perception. In this section we will introduce some simple concepts that can help improve visualization effectiveness. To communicate this message we will start with three questions; what is a visualization, what is human perception, and what makes a good visualization.

What is a visualization?

When portraying outliers, trends, and patterns in data, data visualization is utilized to generate graphical representation of information and data. Over the course of time, a variety of data visualization techniques have emerged, opening up opportunities to find answers to any and all types of analytical questions. For example, line charts can be used to track data over time, bar charts can be used to compare data, and pie charts can be used to explore composition and the breakdown of data. With dozens of chart types available, each known to fit one particular purpose or another, it is extremely important to choose the most applicable type of visualization.

What is human perception?

Human perception is the process of recognizing, organizing, and interpreting sensory information. This includes visual information in the form of images or charts. Human perception is critical when communicating a message visually. Specific elements of data visualization that influence human perception are labeling, ordering, and color association. When labeling a visual, it is important to use precise language that clearly describes the title or axis. Additionally, selectively choose font sizes so that the visual is balanced and flows naturally. When it is desirable to communicate a trend using a chart ordering the data so that the lines are bars are shown descending or ascending will increase the effectiveness of the message. Finally, when selecting colors for the visual, it is necessary to consider how the colors enhance the perception of the message.

What makes a good visualization?

A good visualization utilizes three main components; simplicity, clarity, and creativity. These three components can be used in each step of the creation process. The main steps of the creation process are as follows:

1. Choose the visual that communicates the story the best
2. Create a barebones version of the visual
3. Adjust the colors, sizes, positions, and layers to emphasize the statistic, trend, or area of importance
4. Critique your work. If you answer no to any of the following questions go to step 3 or earlier
    a. Is the visual cluttered?
    b. Is the color scheme effective?
    c. Does the visual communicate the desired message?
    d. Can I make the visual simpler and still communicate the desired message?

Libraries

Matplotlib

Matplotlib is the most popular Python data visualization library. It has been around for a long time (more than 10 years) and works with many platforms including Jupyter notebook. It is a very versatile and easy-to-use library. With just a few lines of code you can generate many types of powerful visuals.

Seaborn

Seaborn is a library that is built on top of Matplotlib providing a greater variety of visualization patterns and themes. Seaborn is specially designed for statistics visualization and provides an easy way to summarize data emphasizing the distribution patterns in the data.

Plotly

Plotly is a sophisticated data visualization tool that is good for creating elaborate plots more efficiently. Although it has many built in capabilities, such as a built-in hover tool for mouse-over actions, it is easily customizable with a fairly intuitive syntax.

Which one should you use?

After determining what type of visual is best for a given situation, you should then choose a library. Matplotlib is best for relatively simple static visuals, like histograms, bar charts, line graphs, etc. so if you want a graph quickly and don't care much about the visuals then matplotlib is sufficient. If you'd like to create an aesthetically pleasing static visual then seaborn is an excellent option which has many of the same capabilities as matplotlib. Finally, if it is desirable to create a sophisticated interactive visualization, plotly would be the library of choice. Sometimes there is a decent bit of work involved but the final product makes all the trouble worth it.

Examples

Each of the following visuals can be created with almost any Python visualization library. Below we will demonstrate when to use each visual with each of the three libraries mentioned above.

Scatter

A scatter plot is a diagram composed of Cartesian coordinates used to display values. Scatter plots are most likely used when you want to determine the correlation between paired numerical data and when your independent variable may have multiple dependent values.

Below is the sample code we will use to generate our scatter plots with each Python library. The dataset used is a subset from the `'covid_daily.csv'` file.

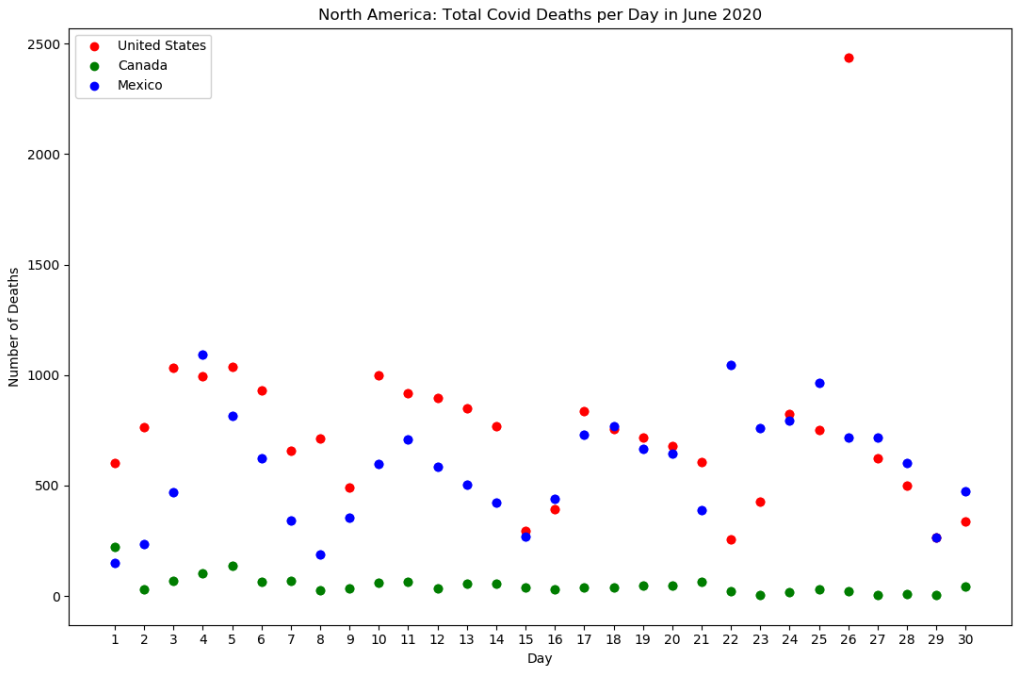
```
>>> import numpy as np
>>> import pandas as pd
>>> data = pd.read_csv('covid_daily.csv')
>>> data = data[['continent', 'location', 'date', 'new_deaths']]
>>> by_continent = data.groupby(['continent', 'location',
'date']).sum().reset_index()
>>> na_stats = by_continent.loc[by_continent['continent'] == 'North
America'].sort_values(by = ['date'])
>>> na_stats['date'] = na_stats['date'].apply(pd.to_datetime)
>>> june_deaths = na_stats[na_stats['date'].between('2020-06-01',
'2020-06-30')].sort_values(by = ['date'])
>>> us = june_deaths.loc[june_deaths['location'] == 'United States']
>>> us.tail(2)
          continent       location       date  new_deaths
27505  North America  United States 2020-06-26      2437.0
27506  North America  United States 2020-06-27       623.0

>>> canada = june_deaths.loc[june_deaths['location'] == 'Canada']
>>> canada.tail(2)
          continent location       date  new_deaths
24255  North America   Canada 2020-06-26        20.0
24256  North America   Canada 2020-06-27         4.0

>>> mexico = june_deaths.loc[june_deaths['location'] == 'Mexico']
>>> mexico.tail(2)
          continent location       date  new_deaths
26112  North America   Mexico 2020-06-26       718.0
26113  North America   Mexico 2020-06-27       719.0
```

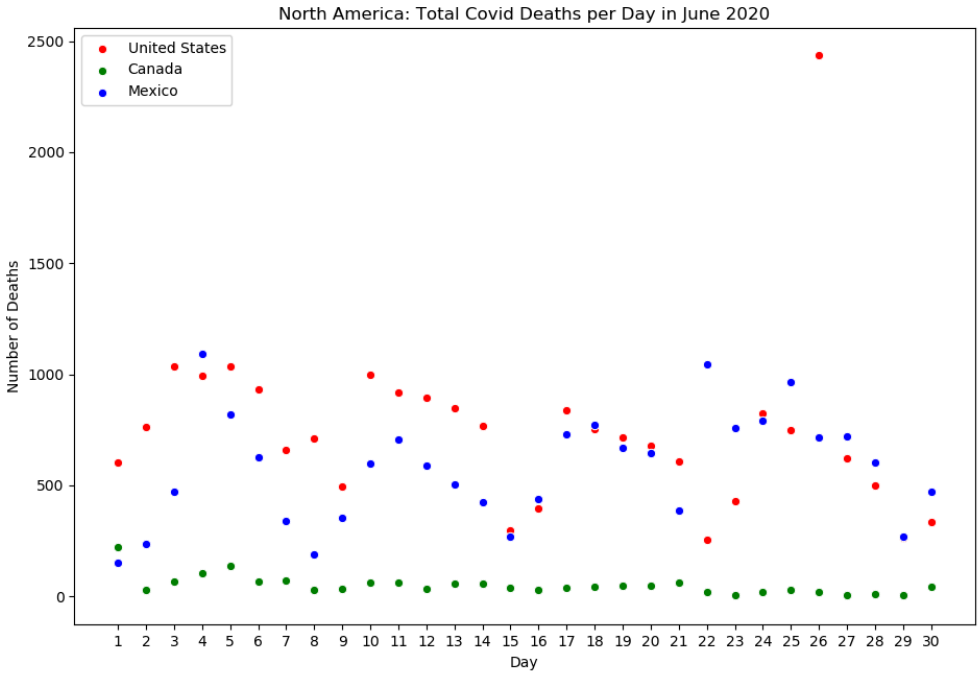
Matplotlib

```
>>> import matplotlib.pyplot as plt
>>> us['date'] = range(1, 31)
>>> canada['date'] = range(1, 31)
>>> mexico['date'] = range(1, 31)
>>> plt.scatter(us['date'], us['new_deaths'], color = 'red', label = 'United
States')
>>> plt.scatter(canada['date'], canada['new_deaths'], color = 'green', label =
'Canada')
>>> plt.scatter(mexico['date'], mexico['new_deaths'], color = 'blue', label =
'Mexico')
>>> plt.xticks(us['date'])
>>> plt.xlabel('Day')
>>> plt.ylabel('Number of Deaths')
>>> plt.title('United States: Total Covid Deaths per Day in June 2020')
>>> plt.legend(loc = 'upper left')
>>> plt.show()
```



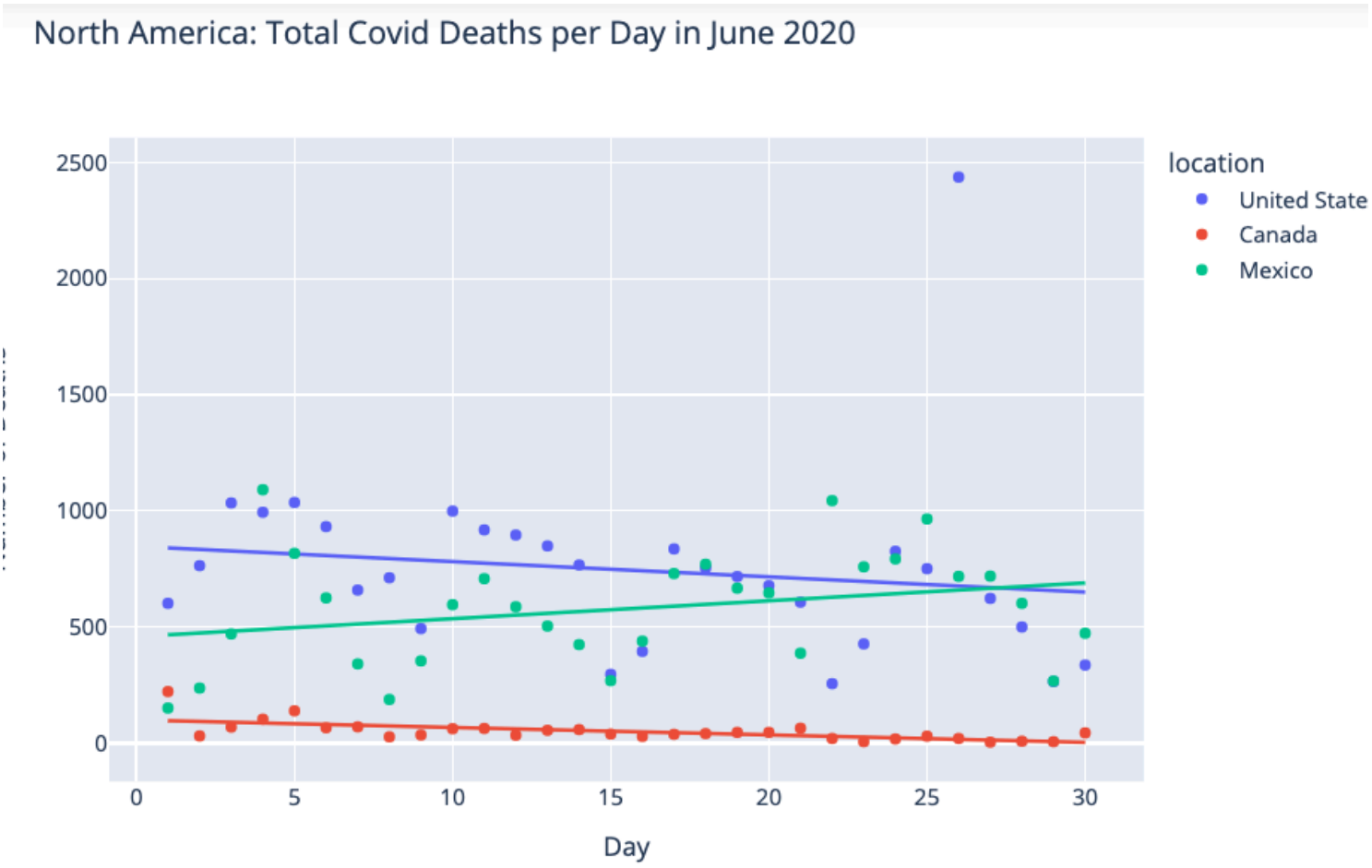
Seaborn

```
>>> import seaborn as sns
>>> import matplotlib.pyplot as plt
>>> us['date'] = range(1, 31)
>>> canada['date'] = range(1, 31)
>>> mexico['date'] = range(1, 31)
>>> ax = sns.scatterplot(x = 'date', y = 'new_deaths', data = us, label = 'United
States', color = 'red')
>>> ax = sns.scatterplot(x = 'date', y = 'new_deaths', data = canada, label =
'Canada', color = 'green')
>>> ax = sns.scatterplot(x = 'date', y = 'new_deaths', data = mexico, label =
'Mexico', color = 'blue')
>>> ax.legend()
>>> plt.xticks(us['date'])
>>> ax.set_xlabel('Day')
>>> ax.set_ylabel('Number of Deaths')
>>> ax.set_title('United States: Total Covid Deaths per Day in June 2020')
>>> plt.show()
```



Plotly

```
>>> import plotly.express as px
>>> us['date'] = range(1, 31)
>>> canada['date'] = range(1, 31)
>>> mexico['date'] = range(1, 31)
>>> combined = pd.concat([us, canada, mexico], ignore_index = True)
>>> fig = px.scatter(combined, x = 'date', y = 'new_deaths', color = 'location',
trendline="ols" , labels = {'date': 'Day', 'new_deaths': 'Number of Deaths'}, title
= 'North America: Total Covid Deaths per Day in June 2020')
>>> fig.show()
```



Line

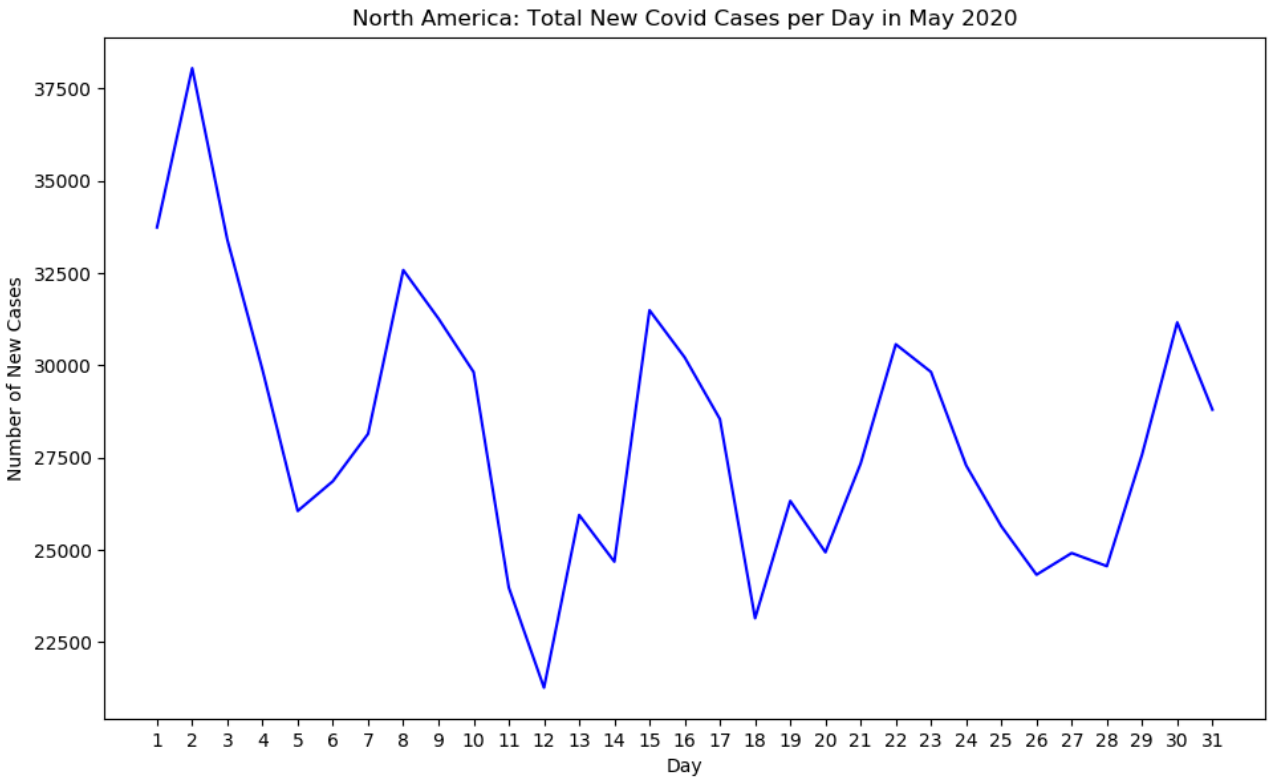
A line chart is a diagram where data is displayed as a series of points interconnected by a line. Line charts are utilized when data is measured periodically to deduce changes and trends between each adjacent value point.

Below is the sample code we will use to generate our scatter plots with each Python library. The dataset used is a subset from the `'covid_daily.csv'` file.

```
>>> import numpy as np
>>> import pandas as pd
>>> data = pd.read_csv('covid_daily.csv')
>>> data = data[['continent', 'location', 'date', 'new_cases']]
>>> by_continent = data.groupby(['continent', 'date']).sum().reset_index()
>>> na_stats = by_continent.loc[by_continent['continent'] == 'North
America'].sort_values(by = ['date'])
>>> na_stats['date'] = na_stats['date'].apply(pd.to_datetime)
>>> cases = na_stats[na_stats['date'].between('2020-05-01',
'2020-05-31')].sort_values(by = ['date'])
>>> cases.tail()
         continent       date  new_cases
741  North America 2020-05-27    24914.0
742  North America 2020-05-28    24559.0
743  North America 2020-05-29    27583.0
745  North America 2020-05-30    31165.0
746  North America 2020-05-31    28801.0
```

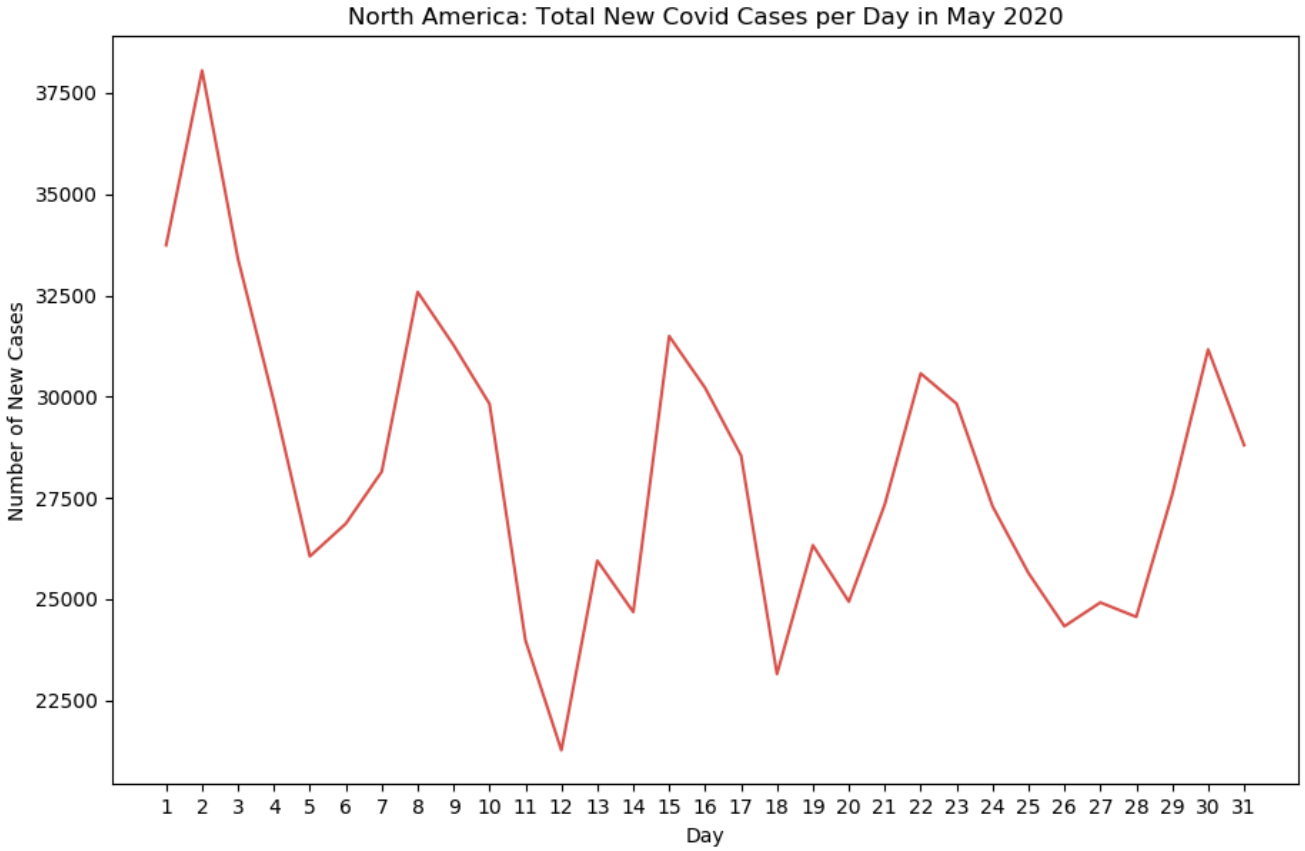
Matplotlib

```
>>> import matplotlib.pyplot as plt
>>> plt.plot(range(1, 32), cases['new_cases'], color = 'blue')
>>> plt.xticks(range(1, 32))
>>> plt.xlabel('Day')
>>> plt.ylabel('Number of New Cases')
>>> plt.title('North America: Total New Covid Cases per Day in May 2020')
>>> plt.show()
```



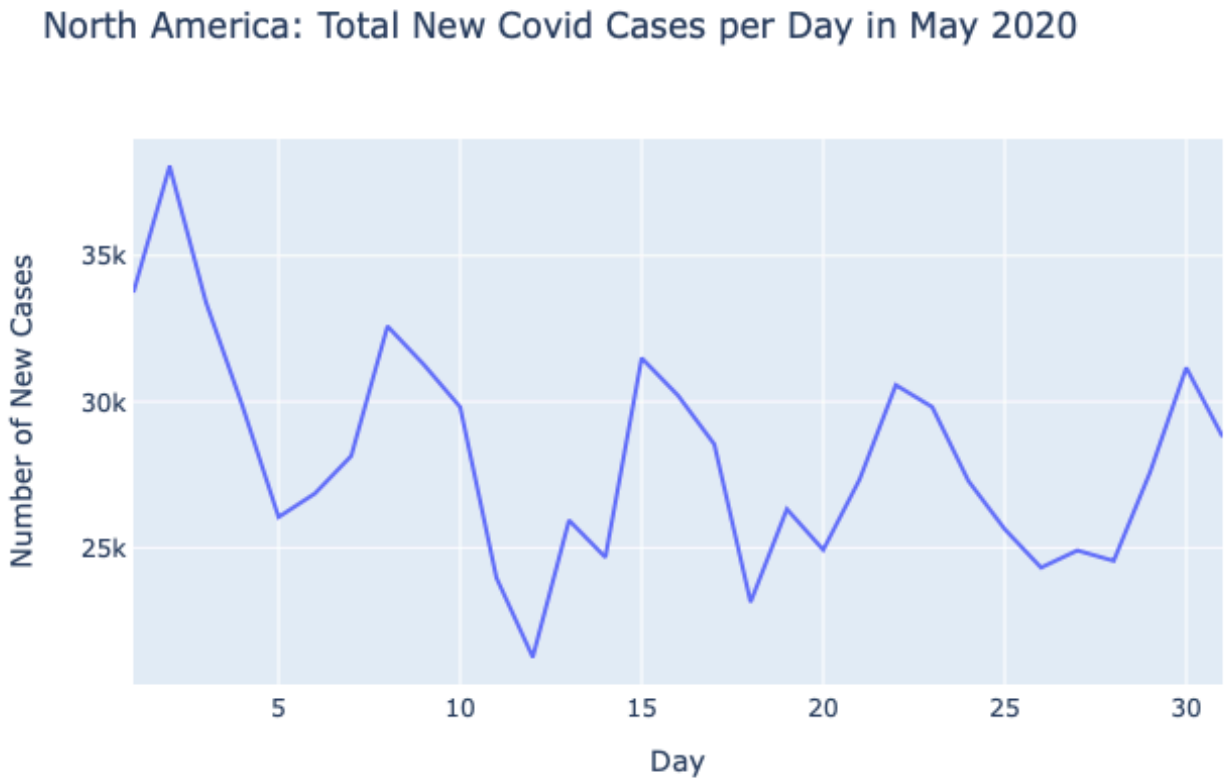
Seaborn

```
>>> import seaborn as sns
>>> import matplotlib.pyplot as plt
>>> ax = sns.lineplot(x = range(1, 32), y = 'new_cases', color = sns.xkcd_rgb['pale
red'],  data = cases)
>>> plt.xticks(range(1, 32))
>>> ax.set_xlabel('Day')
>>> ax.set_ylabel('Number of New Cases')
>>> ax.set_title('North America: Total New Covid Cases per Day in May 2020')
>>> plt.show()
```



Plotly

```
>>> import plotly.express as px
>>> cases['date'] = range(1, 32)
>>> fig = px.line(cases, x = 'date', y = 'new_cases', labels = {'date': 'Day',
'new_cases': 'Number of New Cases'}, title = 'North America: Total New Covid Cases
per Day in May 2020')
>>> fig.show()
```



Bar

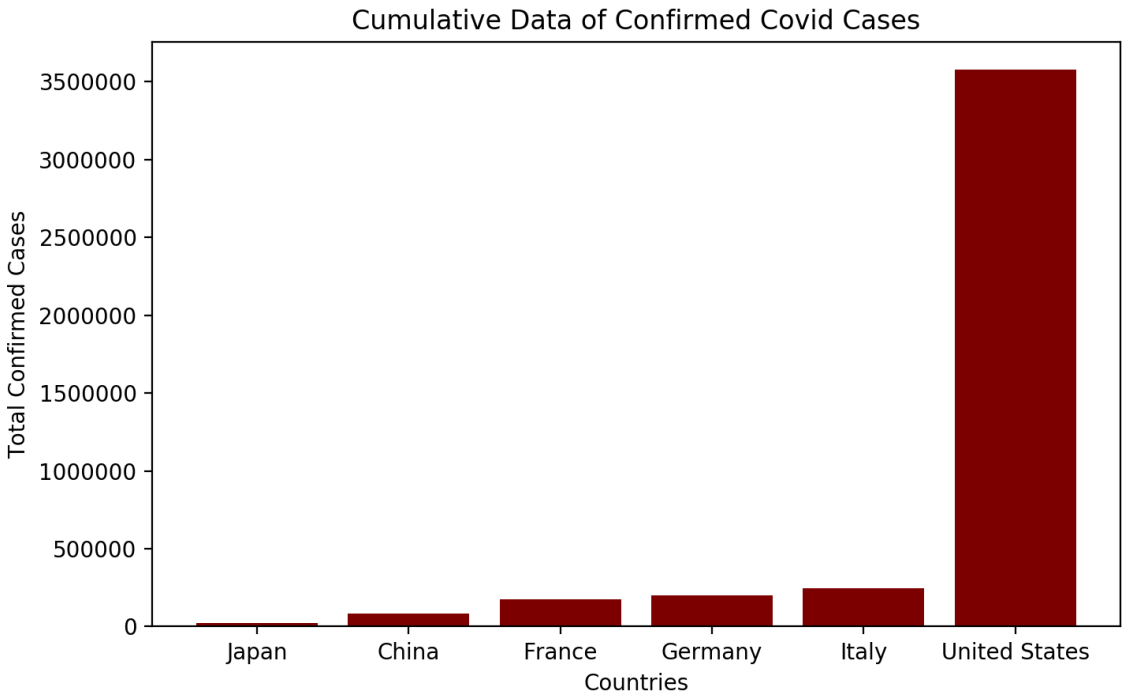
A bar chart is a diagram where categorical data is displayed with bars whose height/length is proportional to the represented values. Bar charts are utilized when you wish to compare categorical data to highlight changes or to show variation in the data set..

Below is the sample code we will use to generate our scatter plots with each Python library. The dataset used is a subset from the `'covid_daily.csv'` file.

```
>>> import numpy as np
>>> import pandas as pd
>>> data = pd.read_csv('covid_daily.csv')
>>> data = data[['location', 'date', 'new_cases', 'new_deaths']]
>>> by_country = data.groupby('location').mean()
>>> countries = by_country.loc[['Japan', 'China', 'United States', 'France',
'Germany', 'Italy']].reset_index().sort_values(by = ['new_cases'])
>>> countries
        location  new_cases
0          Japan    117.365
1          China    426.615
3         France    869.190
4        Germany   1004.215
5          Italy   1218.680
2  United States  17881.105
```

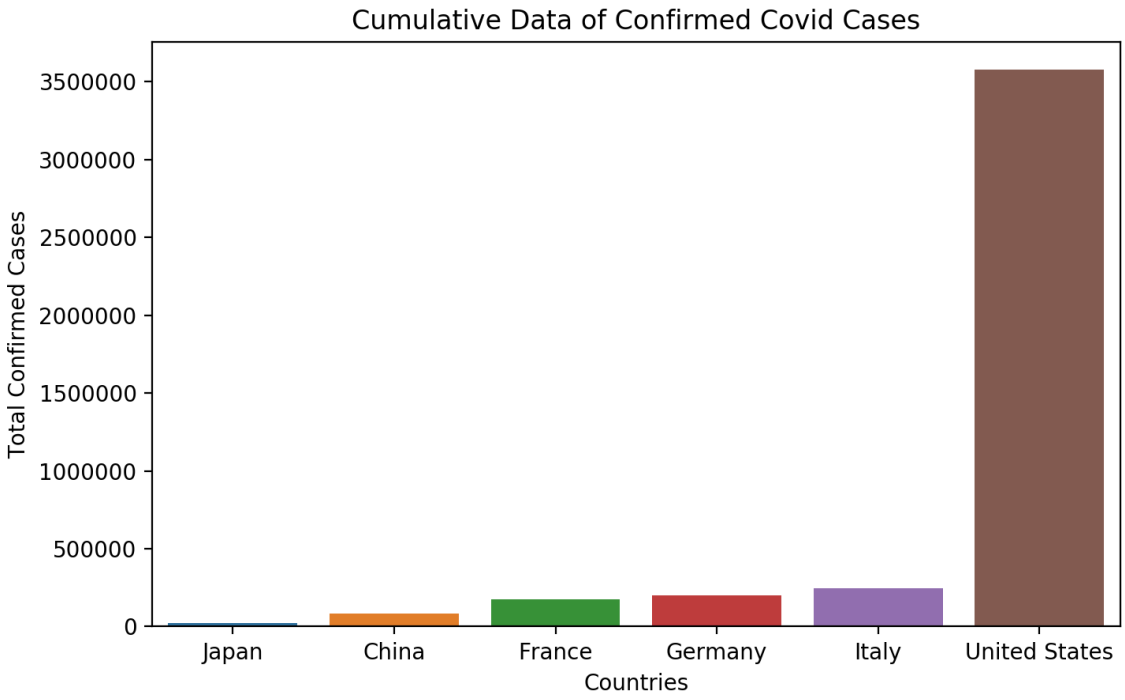
Matplotlib

```
>>> import matplotlib.pyplot as plt
>>> plt.bar(countries['location'], countries['new_cases'], color = 'maroon')
>>> plt.xlabel('Countries')
>>> plt.ylabel('Total Confirmed Cases')
>>> plt.title('Cumulative Data of Confirmed Covid Cases')
>>> plt.show()
```



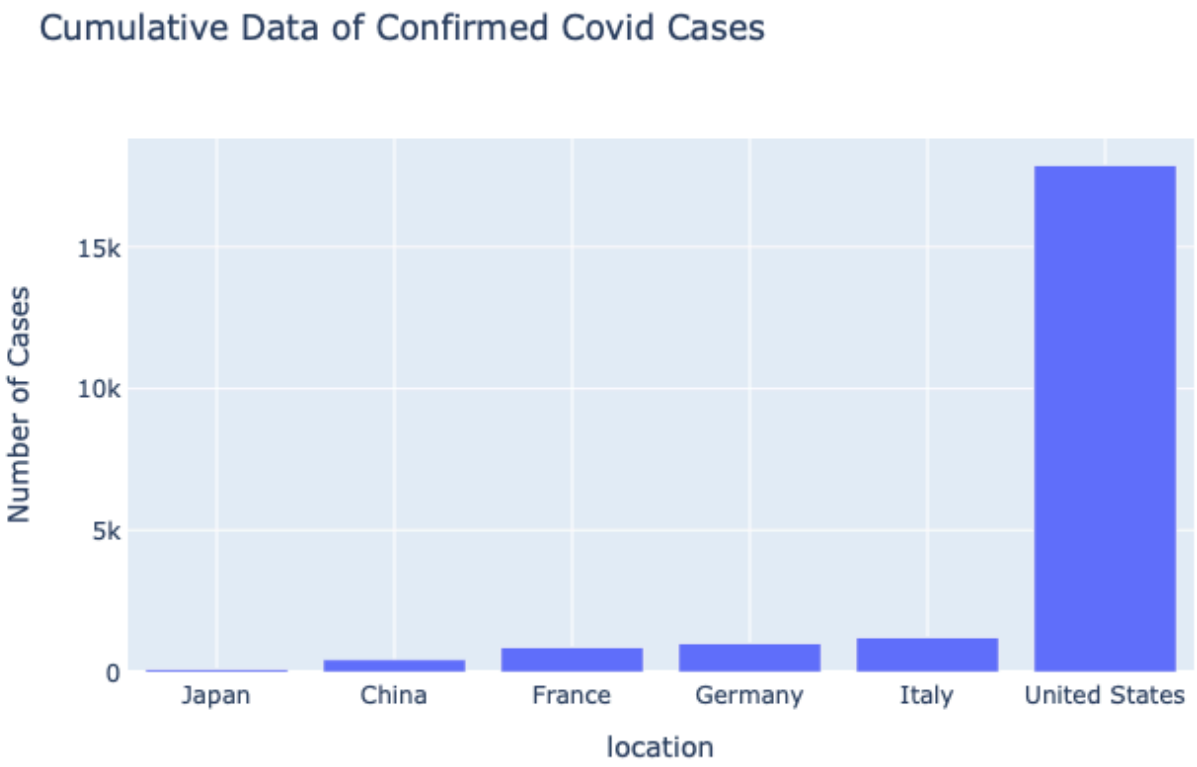
Seaborn

```
>>> import seaborn as sns
>>> import matplotlib.pyplot as plt
>>> ax = sns.barplot(x = 'location', y = 'new_cases', data = countries)
>>> ax.set_xlabel('Countries')
>>> ax.set_ylabel('Total Confirmed Cases')
>>> ax.set_title('Cumulative Data of Confirmed Covid Cases')
>>> plt.show()
```



Plotly

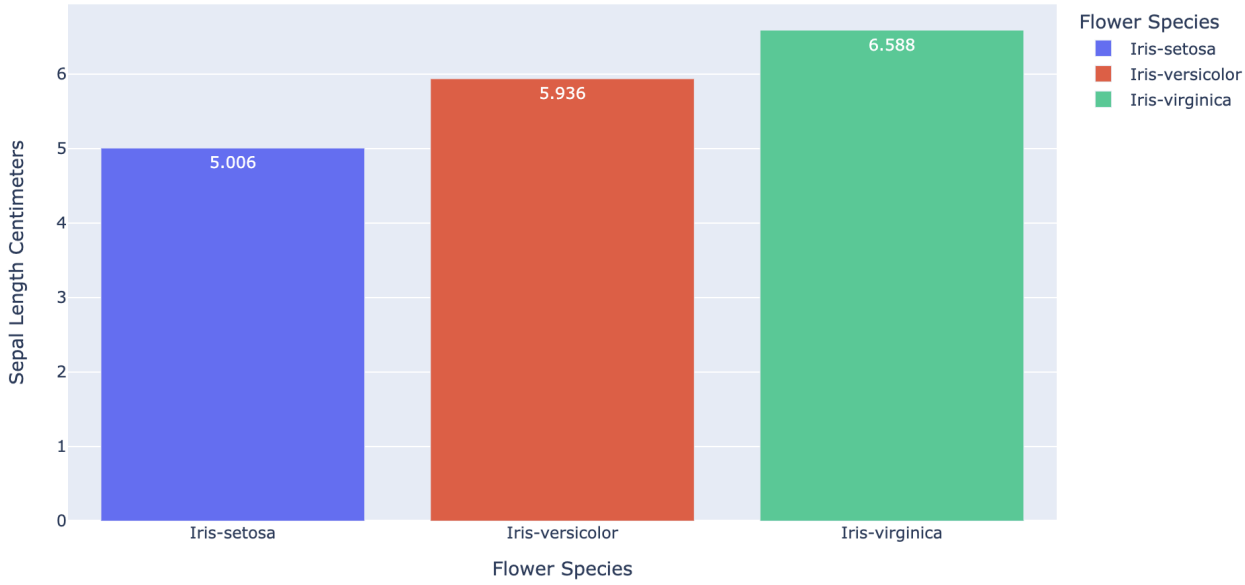
```
>>> import plotly.express as px
>>> fig = px.bar(countries, x = 'location', y = 'new_cases', labels = {'location':
'Countries', 'new_cases': 'Number of Cases'}, title = 'Cumulative Data of Confirmed
Covid Cases')
>>> fig.show()
```



```
>>> length_rates=
iris.groupby('Species')['SepalLengthCm'].mean().reset_index()

>>>fig = px.bar(length_rates, x='Species', y='SepalLengthCm',
color='Species',
             labels={'Species': 'Flower Species'})

>>>fig.update_traces(texttemplate='%{value}',
textposition='auto',textfont_color = 'white')
>>> fig.update_layout(
    yaxis=dict(title='Sepal Length Centimeters'))
```

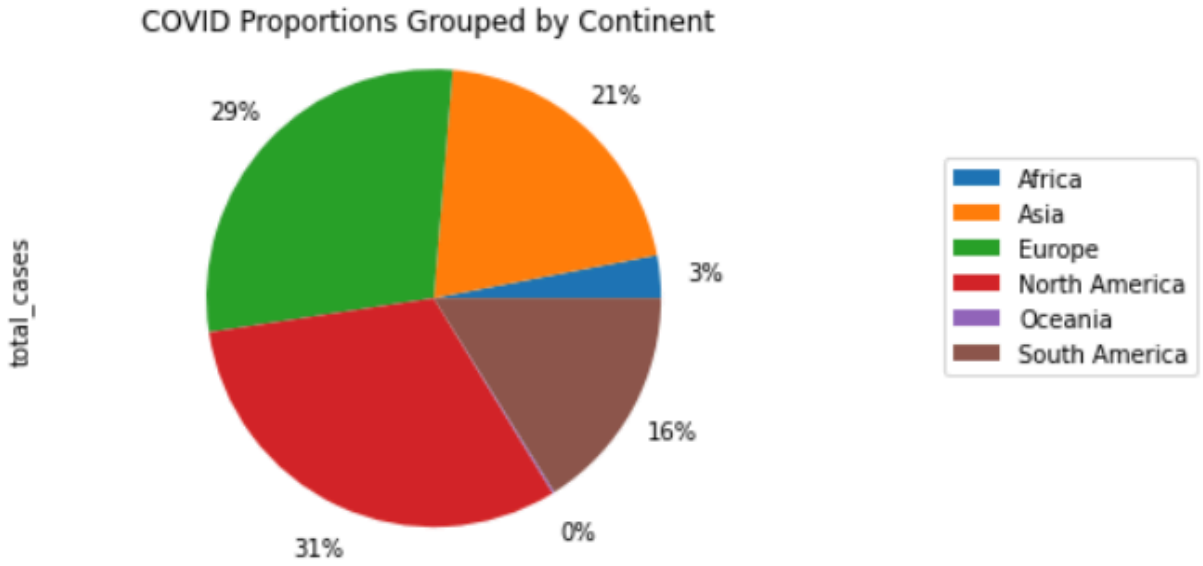


Pie Chart

A pie chart is a type of visualization used to display data as pieces of a whole relative to their numeric value. The size of each individual piece depends on how large the proportion of their numeric value divided by the total sum of numeric values is. Disclaimer: The code below may not show up as intended outside of Jupyter Notebook.

Matplotlib

```
>>> import pandas as pd
>>> import matplotlib.pyplot as plt
>>>
>>> covid_daily = pd.read_csv('covid_daily.csv')
>>> total_case_sum = covid_daily.groupby('continent')['total_cases'].sum()
>>> covid_pie = total_case_sum.plot.pie(labels = None, autopct='%1.0f%%',
pctdistance=1.2)
>>> covid_pie.axis('equal')
>>> covid_pie.set_title("COVID Proportions Grouped by Continent ")
>>> covid_pie.legend(labels=total_case_sum.index, frameon=True,
bbox_to_anchor=(1.5,0.8))
>>> plt.show()
```

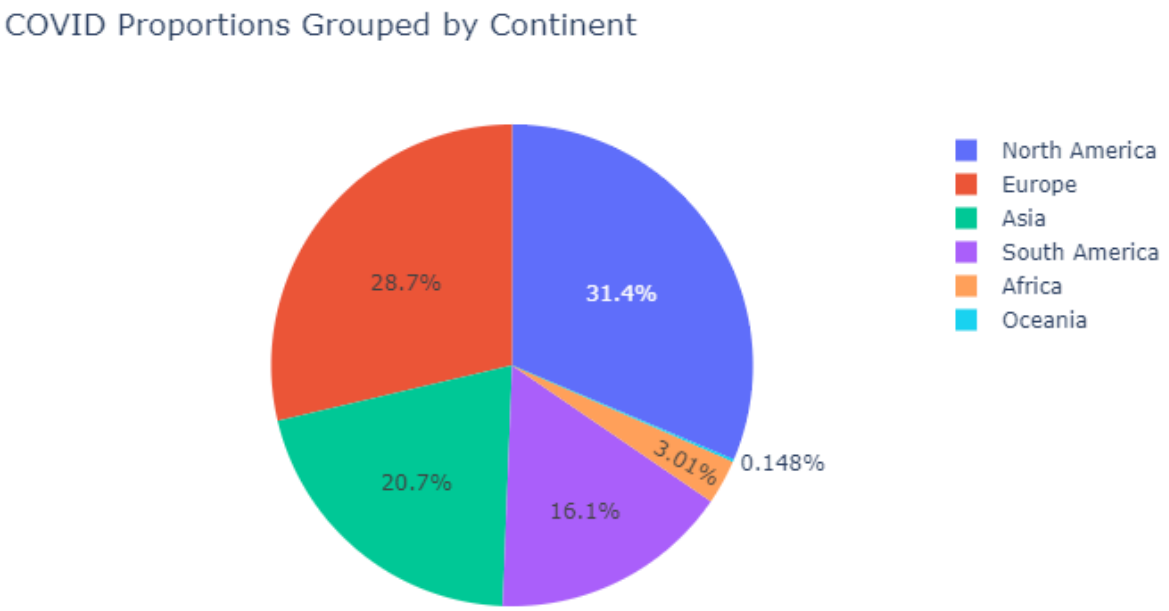


Seaborn

Pie charts have not been implemented in seaborn 0.11.0.

Plotly

```
>>> import plotly.express as px
>>> import pandas as pd
>>>
>>> covid_daily = pd.read_csv('covid_daily.csv')
>>> total_case_sum = covid_daily.groupby('continent')['total_cases'].sum()
>>> fig = px.pie(total_case_sum, values='total_cases', names='total_cases',
title='COVID Proportions Grouped by Continent')
>>> fig.show()
```



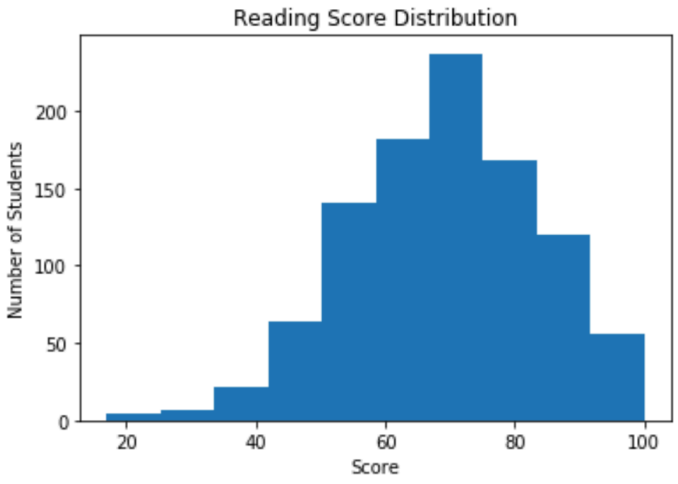
Histogram

A histogram is a type of visualization used to explore and interpret the underlying distribution of a set of data points. A histogram groups the data into logical bins where the number of observations in each bin are compared to the others. Oftentimes when creating a histogram, it is helpful to specify the number of bins to get a better understanding of each group. If too few bins are selected when creating the histogram, then the bins will be wide and could hide important details. Consequently, if too many bins are selected then the bins will be very narrow and could look very crowded and have a lot of noise. A histogram appears to be similar to a bar chart however they're fundamentally different. Bar charts are for categorical data to highlight changes and trends while histograms are for visualizing the distribution of a quantifiable feature.

Below is the sample code we will use to generate our histograms with each Python library. The dataset used can be found at this link. If this link does not work, refer to the dataset here.

Matplotlib

```
>>> df = pd.read_csv("StudentsPerformance.csv")
>>> df.head(2)
   gender race/ethnicity parental level of education  ... math score reading score  writing score
0  female        group B           bachelors degree   ...         72            72             74
1  female        group C                some college  ...         69            90             88
[5 rows x 8 columns]
>>> plt.hist(df["reading score"])
>>> plt.xlabel('Score')
>>> plt.ylabel('Number of Students')
>>> plt.title('Reading Score Distribution')
>>> plt.show()
```



```
>>> from scipy.stats import norm
>>> data = df["reading score"]
>>> mu, sigma = data.mean(), data.std()
>>> n, bins, patches = plt.hist(data, bins=15, density=True)
>>> y = norm.pdf(bins, mu, sigma)
>>> plt.plot(bins, y, 'r--')
>>> plt.xlabel('Score')
>>> plt.ylabel('Probability')
>>> plt.title('Reading Score Distribution')
>>> plt.show()
```

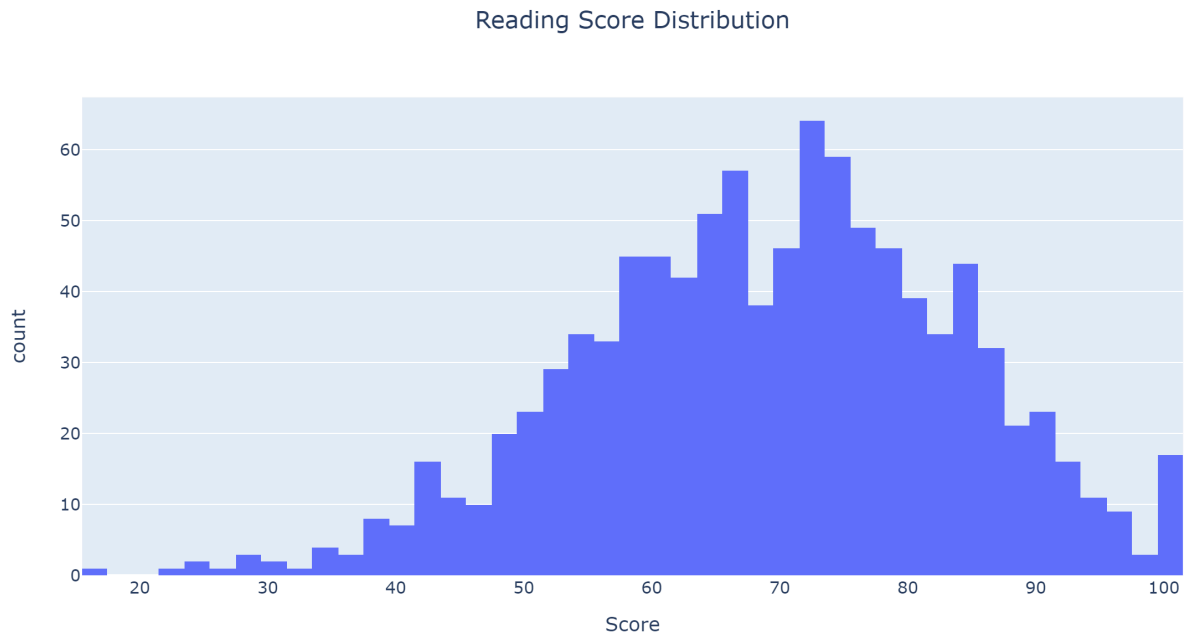
Seaborn

```
>>> sns.set(color_codes = True)
>>> ax = sns.distplot(df["reading score"], kde=False)
>>> ax.set_xlabel('Score')
>>> ax.set_ylabel('Number of Students')
>>> ax.set_title('Reading Score Distribution')
```

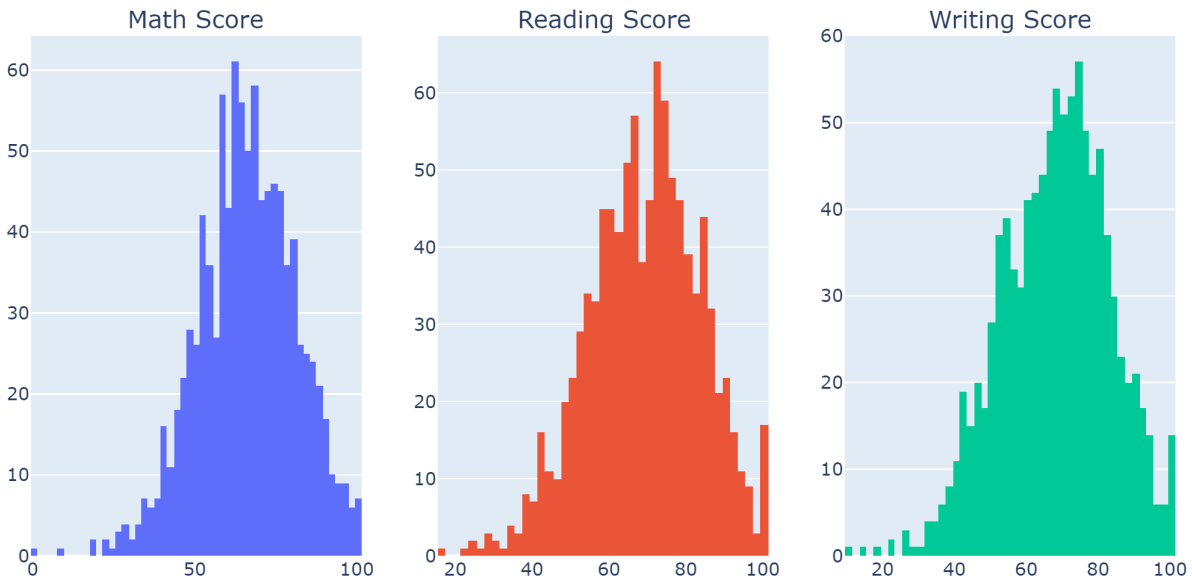

```
>>> from scipy.stats import norm
>>> ax = sns.distplot(df["reading score"], fit = norm)
>>> ax.set_xlabel('Score')
>>> ax.set_ylabel('Probability')
>>> ax.set_title('Reading Score Distribution')
```

Plotly

```
>>> fig = px.histogram(x=df["reading score"], labels={"x" : "Score"},
title="Reading Score Distribution")
>>> fig.update_layout(title_x=0.5)
>>> fig.show()
```

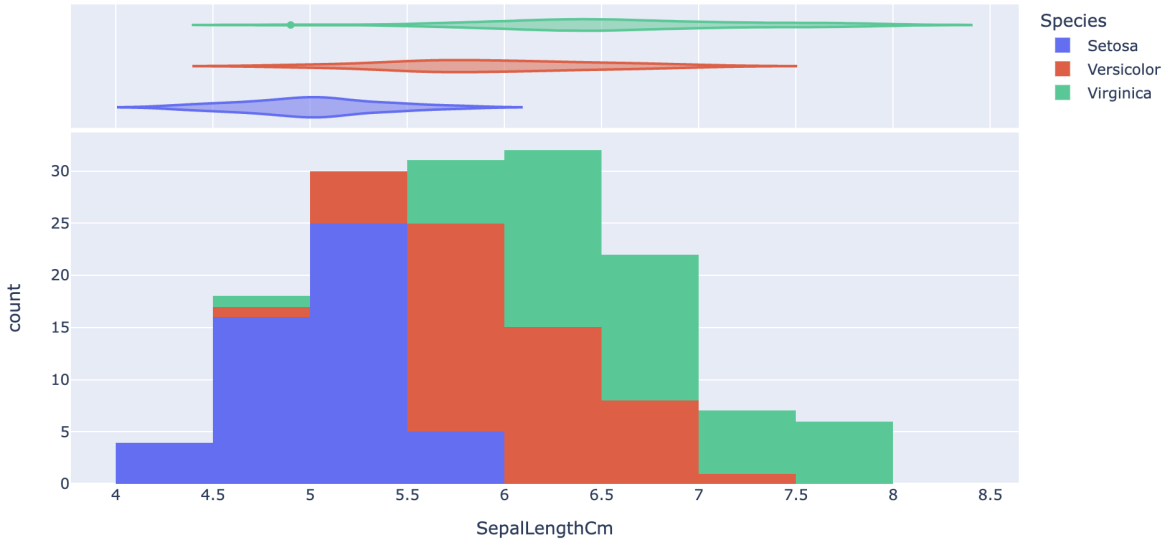


```
>>> import plotly.graph_objects as go
>>> from plotly.subplots import make_subplots
>>> fig = make_subplots(rows=1, cols=3, subplot_titles=["Math
Score","Reading Score","Writing Score"])
>>> trace0 = go.Histogram(x=df["math score"])
>>> trace1 = go.Histogram(x=df["reading score"])
>>> trace2 = go.Histogram(x=df["writing score"])
>>> traces = [trace0, trace1, trace2]
>>> for i, j in enumerate(traces):
...     fig.append_trace(j, 1, i + 1)
>>> fig.show()
```



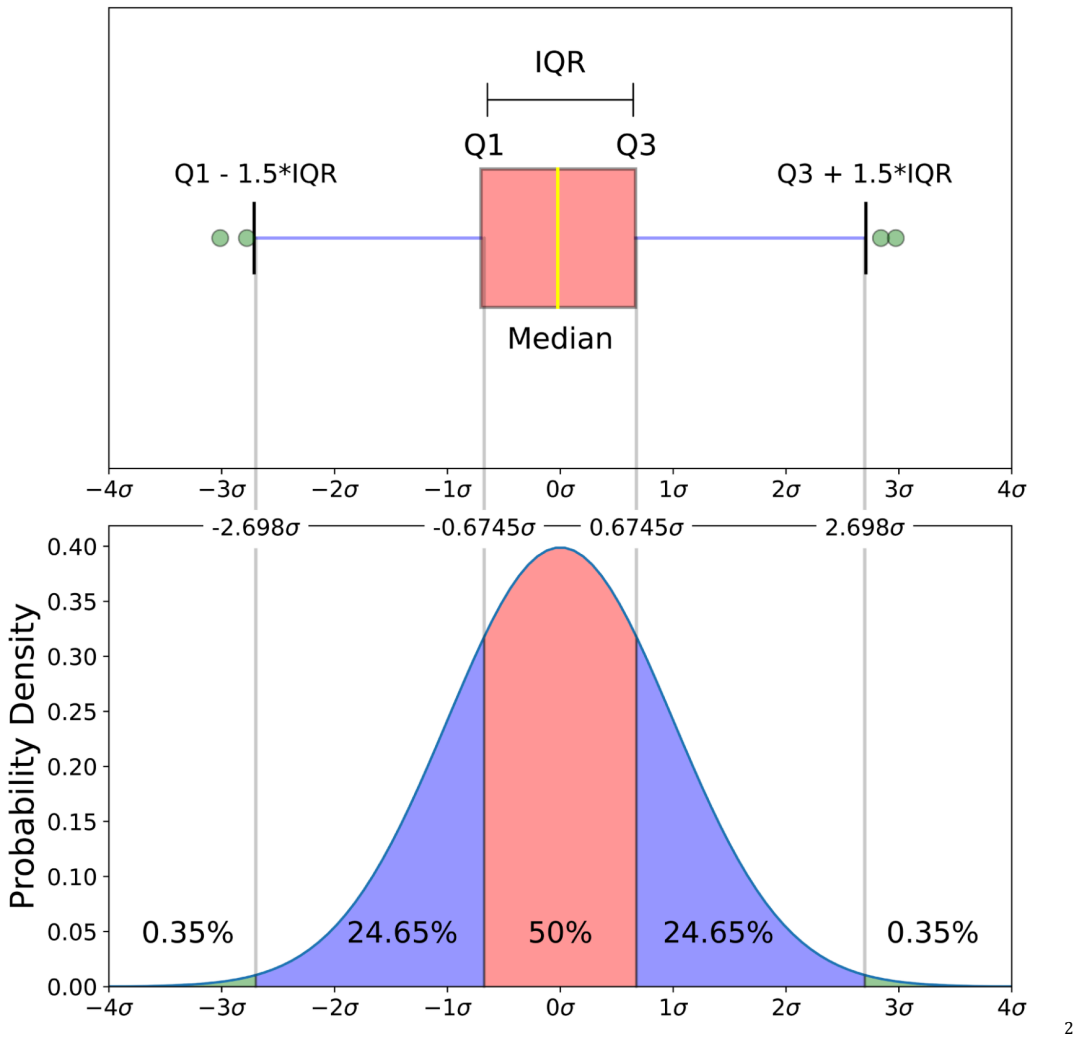
```
>>> fig = px.histogram(iris, x='SepalLengthCm',
          color='Species',
            marginal = 'violin')
>>> original_species = iris['Species']
>>> updated_species = [i.split('-')[-1].capitalize() for i in
original_species]

>>> for original, updated in zip(original_species, updated_species):
    fig.update_traces(name=updated, selector=dict(name=original))
```



Box

Boxplots are visuals commonly seen to conduct descriptive statistical analysis by creating a standardized way of displaying five numerical summaries. The numerical summaries displayed are minimum, first quartile, median (second quartile), third quartile, and maximum. The visual below shows the general structure of a box plot and how it relates to a histogram.



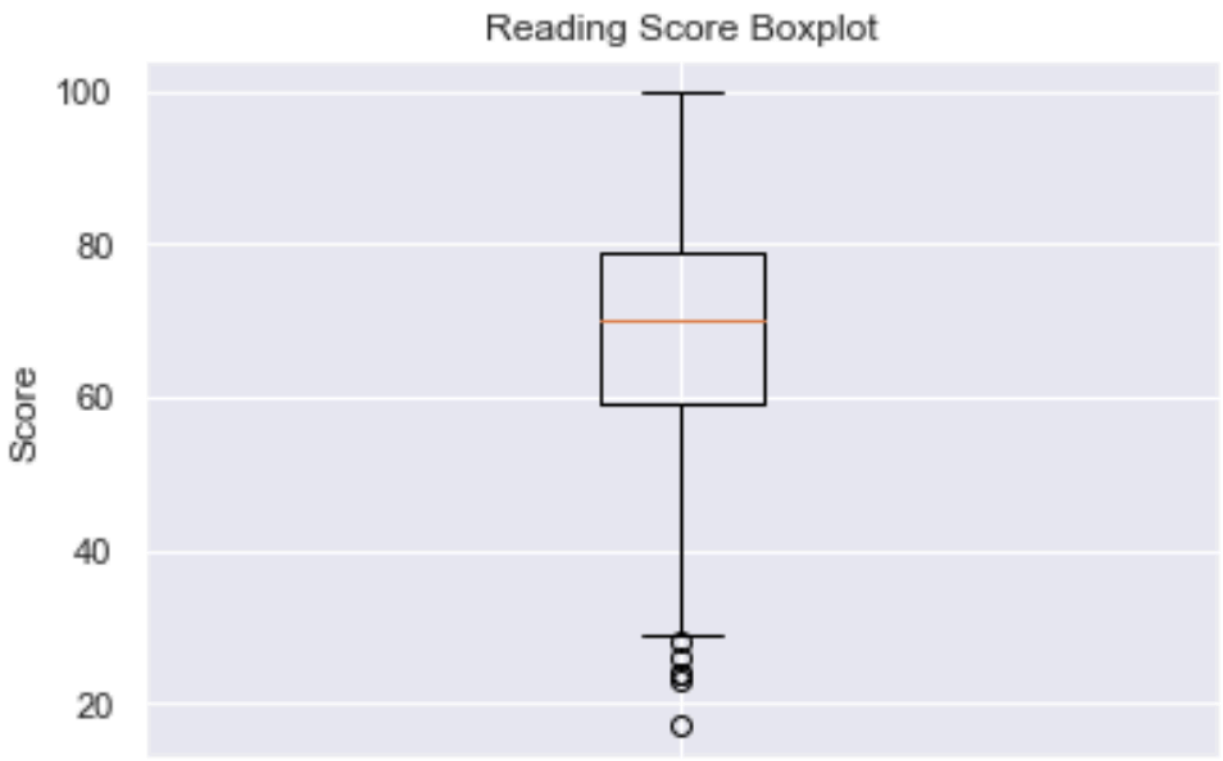
[2]

Below is the sample code we will use to generate our boxplots with each Python library. The dataset used can be found at [this link](#).

[2] Galarnyk, Michael. "Understanding Boxplots." *Medium*, Towards Data Science, 6 July 2020

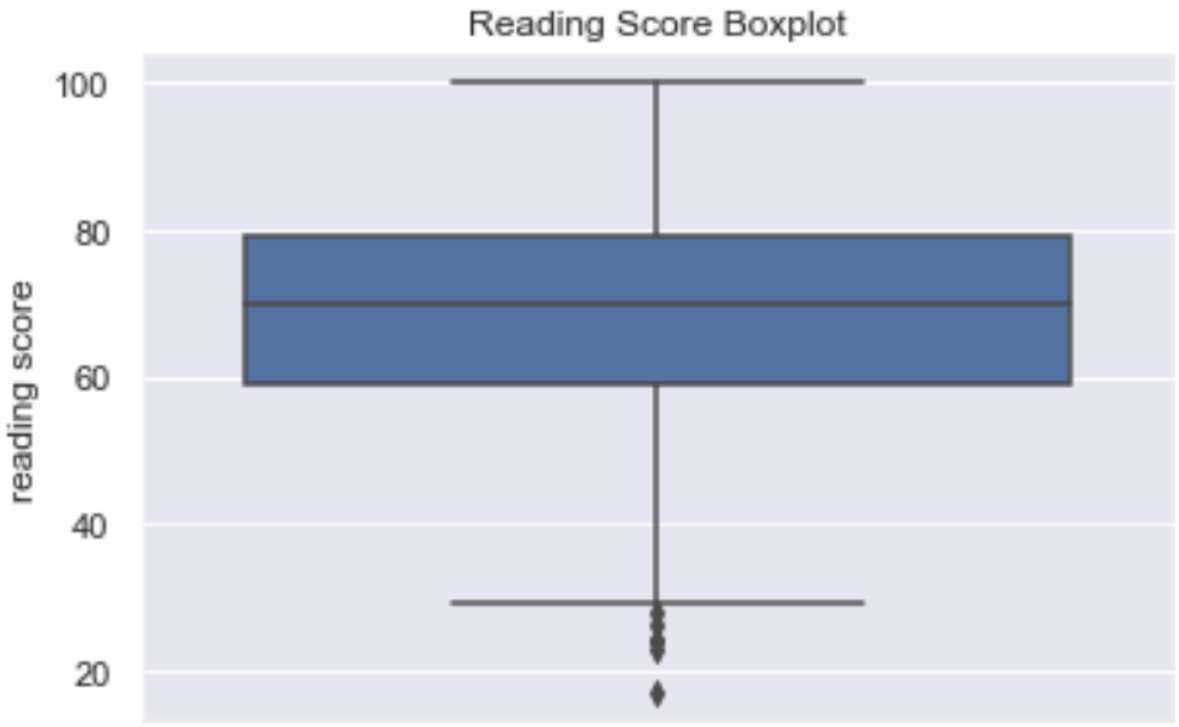
Matplotlib

```
>>> plt.boxplot(df["reading score"])
>>> plt.ylabel('Score')
>>> plt.title('Reading Score Boxplot')
>>> plt.show()
```



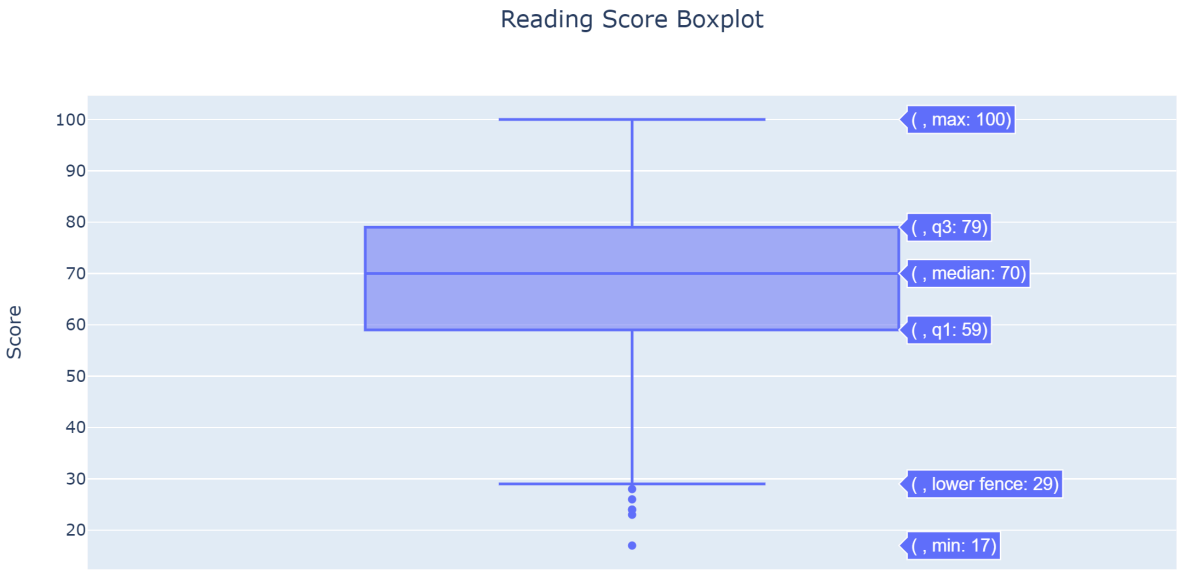
Seaborn

```
>>> ax = sns.boxplot(y=df["reading score"])
>>> ax.set_title('Reading Score Boxplot')
```

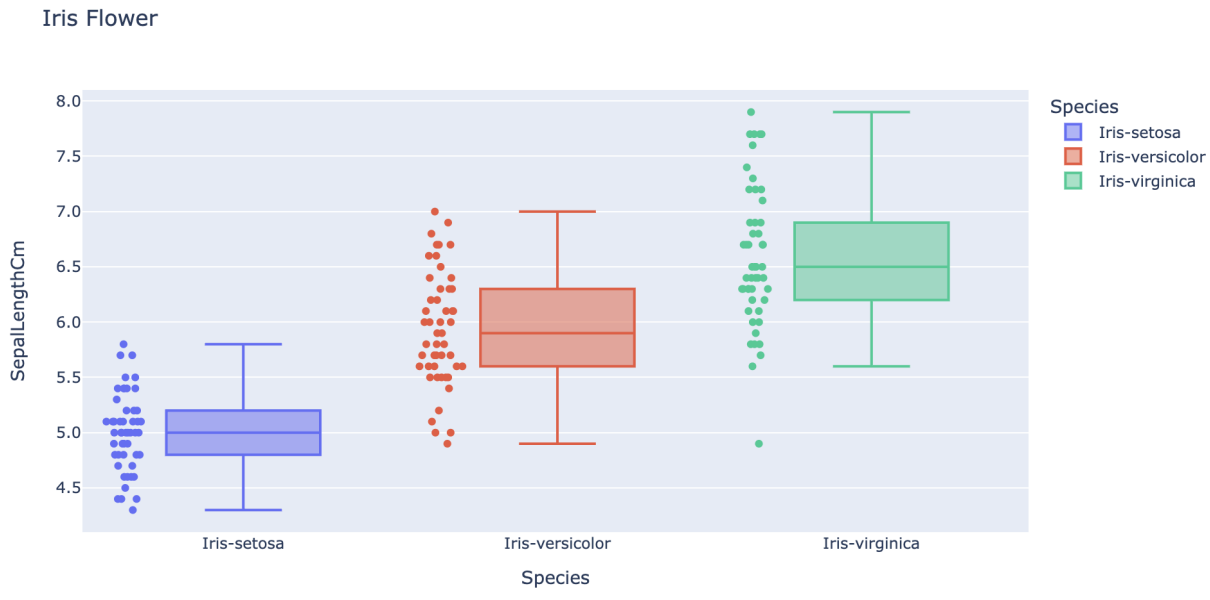


Plotly

```
>>> fig = px.box(y=df["reading score"], labels={"y":"Score"},
title="Reading Score Boxplot")
>>> fig.update_layout(title_x=0.5)
>>> fig.show()
```




```
>>> fig = px.box(iris,x= "Species", y='SepalLengthCm',
              color = 'Species',title='Iris Flower',
              points = 'all')
>>> fig.update_yaxes(tickformat='.1f')
```



More

Word Cloud - A Word cloud is a visual representation of text data. It displays a list of words, the importance of each being shown with font size or color.
Heatmap - A heatmap is a graphical representation of data where the individual values contained in a matrix are represented as colors.
Faceting - Facet plots, also known as trellis plots or small multiples, are figures made up of multiple subplots which have the same set of axes, where each subplot shows a subset of the data
Pair Plot - A pairplot plots pairwise relationships in a dataset. The pairplot function creates a grid of Axes such that each variable in data will be shared in the y-axis across a single row and in the x-axis across a single column.
Choropleth - A Choropleth Map is a map composed of colored polygons. It is used to represent spatial variations of a quantity.